

TP 1.1 — Rotations et conventions

Cours de synthèse à partir de `sim/rotations.py` et `docs/conventions.md`

Projet GNC Quadrotor Outdoor — S0

Sessions des 4, 5 et 6 septembre 2026 (10 h)

Ce document ne contient que ce qui a été dérivé, codé et testé pendant le TP. Rien n'y est ajouté au-delà de `rotations.py` et de `conventions.md` ; les extensions (cinématique d'Euler $\phi, \theta, \psi \rightarrow p, q, r$, méthode de Shepperd, intégration au premier ordre) sont nommées mais renvoyées aux TP qui les traitent.

Sommaire

1	Tableau des notations	2
2	Repères et signes	2
3	Matrices de rotation	3
3.1	Définition et règle des colonnes	3
3.2	Composition	3
3.3	Lacet pur	3
3.4	Forme générale 3-2-1	3
3.5	Matrice antisymétrique	3
4	Quaternions	3
4.1	Les quatre choix de convention (Solà §3)	3
4.2	Produit (Solà éq. 12)	4
4.3	Conjugué, norme, inverse	4
4.4	Rotation d'un vecteur	4
4.5	Axe-angle et double recouvrement	4
4.6	Composition	4
4.7	Quaternion $\rightarrow$ matrice (Solà §2.5)	4
4.8	Matrice $\rightarrow$ quaternion (méthode par la trace)	4
5	Angles d'Euler	5
5.1	Séquence	5
5.2	API	5
5.3	Extraction depuis R	5
5.4	Blocage de cardan	5
6	Intégration de la vitesse angulaire (Solà §4.6)	6
6.1	Un pas sous ω constante	6
6.2	Ce qui est repoussé	6
7	Conventions numériques et batterie de tests	6
7.1	Règles de <code>sim/rotations.py</code>	6
7.2	Ce que chaque test verrouille	7
8	Pièges recensés	7

1 Tableau des notations

Symbole	Signification	Où / remarque
n	repère monde, NED : x = Nord, y = Est, z = Bas	origine locale = <code>ref_lat/lon/alt</code> de PX4
b	repère corps, FRD : x = avant, y = droite, z = bas	flèche de la carte vers l'avant
$\mathbf{v}_n, \mathbf{v}_b$	le <i>même</i> vecteur physique, exprimé dans n ou dans b	indice = repère d'expression
$\mathbf{g}_n$	gravité en NED : $(0, 0, +9,81)$	vers le bas, donc $+z$
$\mathbf{f}$	force spécifique $\mathbf{f} = \mathbf{a} - \mathbf{g}$: ce que <i>mesure</i> un accéléromètre	posé à plat : $(0, 0, -9,81)$ en FRD
$\mathbf{R}_{nb}$	matrice de rotation qui exprime dans n un vecteur donné dans b : $\mathbf{v}_n = \mathbf{R}_{nb} \mathbf{v}_b$	destination en premier
$\mathbf{R}_{bn}$	$= \mathbf{R}_{nb}^\top = \mathbf{R}_{nb}^{-1}$	colonnes orthonormées, $\det = +1$
R_{ij}	coefficient ligne i , colonne j , indices à partir de 0 comme en numpy	$R_{20} = \mathbf{R}[2][0]$
$\mathbf{I}$	identité 3×3	
$\text{tr } \mathbf{R}$	trace = $R_{00} + R_{11} + R_{22}$	
$[\mathbf{a}]_\times$	matrice antisymétrique telle que $[\mathbf{a}]_\times \mathbf{b} = \mathbf{a} \times \mathbf{b}$	<code>skew(a)</code>
$\mathbf{q} = (q_w, \mathbf{q}_v)$	quaternion, ordre (w, x, y, z) , q_w scalaire, $\mathbf{q}_v \in \mathbb{R}^3$	tableau numpy de forme $(4,)$
$\mathbf{q}_{nb}$	quaternion de même sens que $\mathbf{R}_{nb}$: de b vers n	PX4 <code>VehicleAttitude.q</code>
$\mathbf{q}^*$	conjugué $(q_w, -\mathbf{q}_v)$	<code>quaternion_conjugate</code>
$\mathbf{q}^{-1}$	inverse = $\mathbf{q}^* / \ \mathbf{q}\ ^2$; $= \mathbf{q}^*$ ssi $\ \mathbf{q}\ = 1$	pas de fonction inverse
$\otimes$	produit de quaternions (Hamilton, $ij = k$)	<code>quaternion_product</code>
$(0, \mathbf{v})$	quaternion pur : plongement d'un vecteur	partie scalaire nulle
$\mathbf{1}$	quaternion identité $(1, 0, 0, 0)$	$-\mathbf{1}$ code la même rotation
$\alpha, \mathbf{u}$	angle et axe unitaire d'une rotation (axe-angle)	α pour ne pas confondre avec le tangage θ
$\mathbf{q}\{\alpha \mathbf{u}\}$	quaternion de la rotation d'angle α autour de $\mathbf{u}$	<code>build_quaternion(angle, u)</code>
ϕ, θ, ψ	roulis, tangage, lacet (radians)	séquence 3-2-1 intrinsèque, figée
$\mathbf{e}_x, \mathbf{e}_y, \mathbf{e}_z$	vecteurs de base $(1, 0, 0), (0, 1, 0), (0, 0, 1)$	
$\boldsymbol{\omega}_b$	vitesse angulaire du corps, exprimée dans b (le gyro)	rad/s, forme $(3,)$
Δt	pas d'intégration	s
c_ψ, s_ψ	$\cos \psi, \sin \psi$ (idem θ, ϕ)	

2 Repères et signes

- **NED** pour le monde, **FRD** pour le corps. Les deux ont z vers le bas : c'est ce qui rend $\mathbf{g}_n = (0, 0, +9,81)$ positif.

- **Lacet positif** : rotation autour de $+D$ (vers le bas), horaire vu du dessus, nez du Nord vers l'Est.
- **Force spécifique**. Un accéléromètre ne mesure pas la gravité mais $\mathbf{f} = \mathbf{a} - \mathbf{g}$. En chute libre $\mathbf{a} = \mathbf{g}$: il lit 0. Posé sur une table, $\mathbf{a} = 0$: il lit $-\mathbf{g}$, soit $(0, 0, -9,81)$ en FRD, *vers le haut*. C'est la ligne que le modèle de mesure de l'ESKF écrira au TP 2.3 : $\mathbf{f}_{\text{mes}} = \mathbf{R}_{bn}(\mathbf{a}_n - \mathbf{g}_n) + \text{biais} + \text{bruit}$.
- **Actif / passif**. Toutes nos rotations sont *passives* : le vecteur ne bouge pas, on change le repère dans lequel on l'exprime. La formule est la même dans les deux lectures ; c'est la notation $\mathbf{R}_{nb}$ / $\mathbf{q}_{nb}$ (deux repères nommés) qui dit ce qu'on fait.

3 Matrices de rotation

3.1 Définition et règle des colonnes

$\mathbf{R}_{nb}$ transforme les *composantes* : $\mathbf{v}_n = \mathbf{R}_{nb} \mathbf{v}_b$. Ses colonnes sont les axes x_b, y_b, z_b exprimés dans n — c'est la vérification « de tête ». Colonnes orthonormées $\Rightarrow \mathbf{R}^\top \mathbf{R} = \mathbf{I}$, $\det \mathbf{R} = +1$, et donc $\mathbf{R}_{bn} = \mathbf{R}_{nb}^\top$.

3.2 Composition

Les indices s'enchaînent : $\mathbf{R}_{nb} \mathbf{R}_{bc} = \mathbf{R}_{nc}$. Le premier appliqué au vecteur est à *droite*.

3.3 Lacet pur

$$\mathbf{R}_{nb}(\psi) = \begin{pmatrix} c_\psi & -s_\psi & 0 \\ s_\psi & c_\psi & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{R}_{nb}(90^\circ) = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

La version avec $+s_\psi$ en haut à droite est $\mathbf{R}_{bn}$: erreur faite puis corrigée le 6 sept. Test : la première colonne de $\mathbf{R}_{nb}(90^\circ)$ doit être x_b dans NED, c'est-à-dire Est = $(0, 1, 0)$.

3.4 Forme générale 3-2-1

$$\mathbf{R}_{nb}(\phi, \theta, \psi) = \mathbf{R}_z(\psi) \mathbf{R}_y(\theta) \mathbf{R}_x(\phi) = \begin{pmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{pmatrix}.$$

Vérification : $\theta = \phi = 0$ redonne le lacet pur.

3.5 Matrice antisymétrique

$$[\mathbf{a}]_\times = \begin{pmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{pmatrix}, \quad [\mathbf{a}]_\times \mathbf{b} = \mathbf{a} \times \mathbf{b}.$$

Elle est testée contre `np.cross` et réapparaîtra dans tous les jacobiens de l'ESKF.

4 Quaternions

4.1 Les quatre choix de convention (Solà §3)

Choix	Moi	PX4 v1.17	Source
Ordre des composantes	(w, x, y, z)	(w, x, y, z)	<code>VehicleAttitude.msg</code>
Algèbre	Hamilton, $ij = k$	Hamilton	idem
Fonction	passive	passive	« from FRD body to NED earth »
Direction	local $\rightarrow$ global	FRD $\rightarrow$ NED	idem

Le seul test de la batterie qui verrouille Hamilton est $i \otimes j = k$ (JPL donnerait $-k$). Les compositions autour d'un même axe n'y voient rien : leur produit vectoriel est nul.

4.2 Produit (Solà éq. 12)

Avec $\mathbf{p} = (p_w, \mathbf{p}_v)$ et $\mathbf{q} = (q_w, \mathbf{q}_v)$:

$$\mathbf{p} \otimes \mathbf{q} = \begin{pmatrix} p_w q_w - \mathbf{p}_v^\top \mathbf{q}_v \\ p_w \mathbf{q}_v + q_w \mathbf{p}_v + \mathbf{p}_v \times \mathbf{q}_v \end{pmatrix}.$$

Propriétés testées : $\mathbf{q} \otimes \mathbf{1} = \mathbf{1} \otimes \mathbf{q} = \mathbf{q}$; $\|\mathbf{p} \otimes \mathbf{q}\| = \|\mathbf{p}\| \|\mathbf{q}\|$; non commutatif ; deux lacets de 45° donnent $\mathbf{q}_{nb}(90^\circ)$; $i \otimes j = k$.

4.3 Conjugué, norme, inverse

$\mathbf{q}^* = (q_w, -\mathbf{q}_v)$, $\|\mathbf{q}\|^2 = q_w^2 + \mathbf{q}_v^\top \mathbf{q}_v$, $\mathbf{q}^{-1} = \mathbf{q}^* / \|\mathbf{q}\|^2$. On ne manipule que des quaternions unitaires, donc $\mathbf{q}^{-1} = \mathbf{q}^*$ et il n'y a pas de fonction inverse. Test : $\mathbf{q} \otimes \mathbf{q}^* = \mathbf{1}$.

4.4 Rotation d'un vecteur

$$(0, \mathbf{v}_n) = \mathbf{q}_{nb} \otimes (0, \mathbf{v}_b) \otimes \mathbf{q}_{nb}^*.$$

La partie scalaire du résultat est nulle (à l'arrondi près) ; c'est un garde-fou gratuit. $\mathbf{q}_{nb}^* = \mathbf{q}_{bn}$: $\mathbf{q}_{bn} \otimes (0, \mathbf{v}_n) \otimes \mathbf{q}_{bn}^*$ exprime un vecteur monde dans le corps — prédiction des capteurs embarqués. Tests : $\mathbf{q}_{nb}(90^\circ)$ envoie $(1, 0, 0)$ sur $(0, 1, 0)$; norme conservée ; $\mathbf{q}^*$ défait $\mathbf{q}$.

4.5 Axe-angle et double recouvrement

$$\mathbf{q}\{\alpha \mathbf{u}\} = \left(\cos \frac{\alpha}{2}, \mathbf{u} \sin \frac{\alpha}{2} \right), \quad \|\mathbf{u}\| = 1.$$

`build_quaternion` lève `ValueError` si $\mathbf{u} = 0$ (un axe-angle sans axe n'existe pas). Le signe apparaît deux fois dans $\mathbf{q} \otimes (0, \mathbf{v}) \otimes \mathbf{q}^*$: $\mathbf{q}$ et $-\mathbf{q}$ codent la même rotation (Solà §2.4.6). Conséquences : $(-1, 0, 0, 0)$ est l'identité, pas 180° ; une rotation de 180° autour de $\mathbf{u}$ est $(0, \mathbf{u})$; les tests d'aller-retour comparent à $\pm \mathbf{q}$ (`assert_same_rotation`).

4.6 Composition

$\mathbf{R}(\mathbf{q}_1 \otimes \mathbf{q}_2) = \mathbf{R}(\mathbf{q}_1) \mathbf{R}(\mathbf{q}_2)$ (Solà §2.6) : le produit de quaternions se comporte comme le produit de matrices, les indices s'enchaînent, le premier appliqué au vecteur est à droite.

4.7 Quaternion $\rightarrow$ matrice (Solà §2.5)

$$\mathbf{R}(\mathbf{q}) = (q_w^2 - \mathbf{q}_v^\top \mathbf{q}_v) \mathbf{I} + 2 \mathbf{q}_v \mathbf{q}_v^\top + 2 q_w [\mathbf{q}_v]_\times.$$

Le premier terme est un scalaire fois l'identité (il vaut $\cos \alpha$ pour un quaternion unitaire, pas 1) ; le deuxième est un produit *extérieur* (`np.outer`, pas `@`) ; le troisième la matrice antisymétrique. Tests : $\mathbf{q}_{nb}(90^\circ) \mapsto \mathbf{R}_{nb}(90^\circ)$ en dur ; $\mathbf{R} \mathbf{R}^\top = \mathbf{I}$ et $\det \mathbf{R} = 1$; $\mathbf{R} \mathbf{v}$ égal à la rotation par quaternion (formule contre définition).

4.8 Matrice $\rightarrow$ quaternion (méthode par la trace)

Dérivée de $\mathbf{R}(\mathbf{q})$ avec $\|\mathbf{q}\| = 1$:

$$\begin{aligned} \text{tr } \mathbf{R} = 4q_w^2 - 1 & \Rightarrow q_w = \sqrt{\frac{\text{tr } \mathbf{R} + 1}{4}}, \\ \mathbf{R} - \mathbf{R}^\top = 4q_w [\mathbf{q}_v]_\times & \Rightarrow q_x = \frac{R_{21} - R_{12}}{4q_w}, \quad q_y = \frac{R_{02} - R_{20}}{4q_w}, \quad q_z = \frac{R_{10} - R_{01}}{4q_w}. \end{aligned}$$

Deux choix cachés, écrits dans `conventions.md` :

- la racine positive impose $q_w \geq 0$ en sortie : le double recouvrement est levé par convention ;
- la division par q_w explose vers $\alpha = 180^\circ$. Garde-fou **avant** la racine : **ValueError** si $\text{tr } \mathbf{R} + 1 < 4 \cdot 10^{-6}$, soit $q_w < 10^{-3}$, soit $|\alpha - 180^\circ| < 0,11^\circ$. Pas de méthode de Shepperd : la fonction sert aux tests et à des conversions ponctuelles, PX4 livre des quaternions.

Pourquoi *avant* la racine : à 180° , $\text{tr } \mathbf{R} + 1$ vaut $-2 \cdot 10^{-16}$ dans un cas sur cinq ; `np.sqrt` rend alors `nan` avec un simple `RuntimeWarning`, et `np.isclose(nan, 0)` est faux. Le premier garde-fou, placé après la racine, laissait passer des quaternions de `nan` en silence.

5 Angles d'Euler

5.1 Séquence

3-2-1 intrinsèque (Z-Y-X), identique à PX4 `src/lib/matrix/matrix/Euler.hpp` : lacet ψ autour de z du NED, puis tangage θ autour du y du repère obtenu, puis roulis ϕ autour du x du repère obtenu (le x corps).

$$\mathbf{R}_{nb} = \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\phi), \quad \mathbf{q}_{nb} = \mathbf{q}_z(\psi) \otimes \mathbf{q}_y(\theta) \otimes \mathbf{q}_x(\phi),$$

avec $\mathbf{q}_z(\psi) = \mathbf{q}\{\psi\mathbf{e}_z\}$, etc. Même ordre pour les quaternions que pour les matrices, par la propriété de composition. Lecture équivalente : extrinsèque X-Y-Z (rotations successives autour des axes *fixes* x , puis y , puis z) — même produit de matrices.

5.2 API

`euler_to_quaternion(phi, theta, psi)` et `quaternion_to_euler(q)` $\rightarrow$ `[phi, theta, psi]` : trois scalaires en radians, ordre roulis-tangage-lacet (celui de PX4). La séquence n'est pas un paramètre.

5.3 Extraction depuis R

En lisant la forme générale du §3.4 :

$$\theta = -\arcsin R_{20}, \quad \phi = \text{atan2}(R_{21}, R_{22}), \quad \psi = \text{atan2}(R_{10}, R_{00}).$$

- `np.clip` de R_{20} à $[-1, 1]$ avant l'`arcsin` : par arrondi, R_{20} peut valoir `1,0000000000000002` et l'`arcsin` rendrait `nan`.
- `atan2` sur un *couple*, pas `arctan` d'un quotient : le quadrant est conservé (pour $\psi = 135^\circ$, $\arctan(R_{10}/R_{00}) = -45^\circ$) et il n'y a pas de division par 0. Le facteur c_θ commun aux deux coefficients est positif sur la branche de l'`arcsin` et ne change pas le quadrant.
- Plages : $\theta \in [-\pi/2, \pi/2]$, $\phi, \psi \in (-\pi, \pi]$.

5.4 Blocage de cardan

À $|\theta| = 90^\circ$, $c_\theta = 0$ et les quatre coefficients $R_{00}, R_{10}, R_{21}, R_{22}$ sont nuls : `atan2(0, 0)` n'est pas défini. Physiquement, nez à la verticale, l'axe de roulis (x corps) et l'axe de lacet (z monde) sont confondus ; seule $\phi \pm \psi$ est définie. C'est la raison de garder $\mathbf{q}$ dans l'estimateur et de réserver les Euler à l'affichage. Le test d'aller-retour borne θ à $\pm(\pi/2 - 0,1)$.

6 Intégration de la vitesse angulaire (Solà §4.6)

6.1 Un pas sous ω constante

Si le corps tourne à ω_b constante pendant Δt , il a tourné de l'angle $\|\omega_b\|\Delta t$ autour de l'axe $\omega_b/\|\omega_b\|$:

$$\mathbf{q}_{n+1} = \mathbf{q}_n \otimes \mathbf{q}\{\omega_b \Delta t\}, \quad \mathbf{q}\{\omega_b \Delta t\} = \text{build_quaternion}(\|\omega_b\|\Delta t, \omega_b).$$

- L'incrément est **à droite** : ω_b est mesurée dans b , c'est une perturbation *locale*, et une rotation locale se compose à droite (règle « le premier appliqué au vecteur est à droite », Solà §4.1). Le test « lacet de 90° puis roulis de 90° autour du x corps, résultat Euler($\pi/2, 0, \pi/2$) » tombe si l'incrément est du mauvais côté.
- Cette formule est *exacte* par pas (ordre zéro, bloqueur d'ordre zéro) : c'est ce qui permet le test « N pas de Δt = un pas de $N\Delta t$ ». Sur un vol réel, ω n'est pas constante ; on appelle la fonction à chaque échantillon gyro avec la nouvelle mesure, et l'erreur vient de la variation de ω pendant Δt (ordre Δt^2 par pas). C'est aussi le noyau de `predictState` d'EKF2.
- Court-circuit : si $\|\omega_b\|\Delta t = 0$, la fonction renvoie $\mathbf{q}$ tel quel (vol stationnaire), sans appeler `build_quaternion`.
- **Seule fonction qui renormalise**, en sortie : $\mathbf{q}_{n+1} \leftarrow \mathbf{q}_{n+1}/\|\mathbf{q}_{n+1}\|$. Raison : c'est la seule fonction rebouclée sur son propre résultat ; les erreurs de 10^{-16} des produits s'y accumulent à 200 Hz. Les autres fonctions partent d'entrées unitaires et rendent une sortie unitaire à l'arrondi près.

6.2 Ce qui est repoussé

Le raffinement au premier ordre de Solà §4.6.2, $\mathbf{q}_{n+1} \approx \mathbf{q}_n \otimes (\mathbf{q}\{\bar{\omega}\Delta t\} + \frac{\Delta t^2}{24}(0, \omega_n \times \omega_{n+1}))$, corrige deux effets d'un ω variable entre deux échantillons : $\bar{\omega}$ la variation de norme, $\omega_n \times \omega_{n+1}$ la rotation de l'axe. Les deux sont nuls sous ω constante. Candidat TP 2.3, seulement si la comparaison aux logs le justifie.

7 Conventions numériques et batterie de tests

7.1 Règles de `sim/rotations.py`

- Formes : quaternion (4,), vecteur (3,), matrice (3, 3). Jamais (n, 1). Avec des 1-D : $\mathbf{a} @ \mathbf{b}$ = produit scalaire, `np.outer(a, b)` = $\mathbf{a}\mathbf{b}^\top$, $\mathbf{M} @ \mathbf{a}$ = $\mathbf{M}\mathbf{a}$, $\mathbf{a} @ \mathbf{M}$ = $\mathbf{M}^\top \mathbf{a}$; `.T` est sans effet sur un 1-D.
- Une fonction ne modifie jamais ses arguments : `q[1:]` est une *vue*, écrire dedans écrit dans le tableau de l'appelant. `q.copy()` d'abord.
- Erreurs : `raise ValueError` avec la cause. Jamais `assert` dans une fonction (`AssertionError`, supprimé par `python -O`) ; les `assert` vivent dans les tests.
- `np.sqrt` d'un négatif et `x / 0` ne lèvent pas : `nan` et `RuntimeWarning`. Tester la quantité *avant* l'opération, avec un seuil choisi et écrit en commentaire avec sa signification physique.
- Égalité de flottants : `np.isclose` / `np.allclose`, jamais `==`.

7.2 Ce que chaque test verrouille

Test	Ce qu'il attrape
$\mathbf{q} \otimes \mathbf{1} = \mathbf{q}$	un produit qui ne fait rien
$\ \mathbf{p} \otimes \mathbf{q}\ = 1$	une erreur de signe ou de terme dans le produit
$\mathbf{p} \otimes \mathbf{q} \neq \mathbf{q} \otimes \mathbf{p}$	un produit vectoriel oublié
$i \otimes j = k$	la convention Hamilton (seul test qui la voit)
deux lacets $45^\circ = \mathbf{q}_{nb}(90^\circ)$	la composition (aveugle au signe du produit vectoriel)
$\mathbf{q} \otimes \mathbf{q}^* = \mathbf{1}$	le conjugué
$\mathbf{q}_{nb}(90^\circ) : (1, 0, 0) \mapsto (0, 1, 0)$	le sens de la rotation (actif/passif, $\mathbf{R}_{nb}$ vs $\mathbf{R}_{bn}$)
$[\mathbf{a}]_{\times} \mathbf{b} = \mathbf{a} \times \mathbf{b}$	chaque coefficient et signe de la matrice antisymétrique
$\mathbf{R}_{nb}(90^\circ)$ en dur	une transposée dans $\mathbf{R}(\mathbf{q})$
$\mathbf{R}\mathbf{R}^\top = \mathbf{I}$, $\det \mathbf{R} = 1$	la structure (une réflexion passerait le premier seul)
$\mathbf{R}\mathbf{v} = \text{rotate}(\mathbf{q}, \mathbf{v})$	un coefficient faux dans $\mathbf{R}(\mathbf{q})$
aller-retour $\mathbf{q} \rightarrow \mathbf{R} \rightarrow \pm \mathbf{q}$	$\mathbf{R} \rightarrow \mathbf{q}$ dans le domaine
rejet à 180° , axes aléatoires	le garde-fou (l'axe $(1, 1, 1)/\sqrt{3}$ passait par chance)
oracle <code>scipy.from_euler("ZYX", [\psi, \theta, \phi])</code>	la séquence d'Euler et l'ordre du produit
aller-retour Euler, $ \theta < \pi/2 - 0,1$	l'extraction, hors cardan
lacet puis roulis corps = <code>Euler(\pi/2, 0, \pi/2)</code>	le côté de l'incrément dans l'intégration
N pas = un pas	l'exactitude sous ω constante
norme après 10^4 pas	le contrat de renormalisation (effet visible en float32 seulement)
$\omega = 0 \Rightarrow \mathbf{q}$ inchangé	le court-circuit

Réflexe acquis : un test vert peut l'être pour une mauvaise raison (`if/print` inversé ; test de forme qui passait par l'identité mal formée ; garde-fou vérifié sur un axe où l'arrondi tombait juste). Se demander si le test passerait avec une fonction fausse.

8 Pièges recensés

- `scipy : as_quat()` sort du (x, y, z, w) ; `from_euler("ZYX", [psi, theta, phi])`, majuscules = intrinsèque, angles *dans l'ordre des lettres* (l'inverse de notre API) ; "xyz" minuscules donne la même matrice, "zyx" non. Comparer via `.as_matrix()`.
- Hamilton vs JPL ; (w, x, y, z) vs (x, y, z, w) .
- $\mathbf{R}_{nb}$ vs $\mathbf{R}_{bn}$: la transposée est l'erreur la plus silencieuse (faite une fois, sur le lacet ψ).
- $(-1, 0, 0, 0)$ n'est pas 180° .
- Doc et sources PX4 : v1.17, jamais `main`.

9 Questions de contrôle (niveau entretien)

À savoir répondre sans notes ; les réponses sont toutes dans ce document.

1. Tourner un vecteur avec un quaternion : la formule, et le sens de $\mathbf{q}_{nb}$ dans nos notations.
2. Actif ou passif : qu'est-ce qui distingue les deux, si la formule est la même ?
3. Pourquoi $\mathbf{q}$ et $-\mathbf{q}$ codent la même rotation ? Que vaut alors $(-1, 0, 0, 0)$?
4. Quel est le seul test qui distingue Hamilton de JPL, et pourquoi les compositions autour d'un même axe n'y voient rien ?
5. Que mesure un accéléromètre posé à plat, composante par composante en FRD ? Et en chute libre ?

6. Pourquoi `atan2` sur un couple plutôt que `arcsin` ou `arctan` d'un quotient ? (deux raisons)
7. Où est le blocage de cardan, que signifie-t-il physiquement, et quelle décision d'architecture en découle ?
8. Pourquoi l'incrément d'intégration est-il à droite de $\mathbf{q}_n$?
9. « Sous ω constante » : constante sur quoi ? Où est l'approximation sur un vol réel ?
10. Pourquoi renormaliser dans l'intégration et nulle part ailleurs ?
11. Le garde-fou de $\mathbf{R} \rightarrow \mathbf{q}$: sur quelle quantité, quel seuil, quel angle, et pourquoi *avant* la racine ?
12. Que vaut $\mathbf{a} \odot \mathbf{b}$ pour deux tableaux (3,) ? Et comment obtenir $\mathbf{a}\mathbf{b}^\top$?